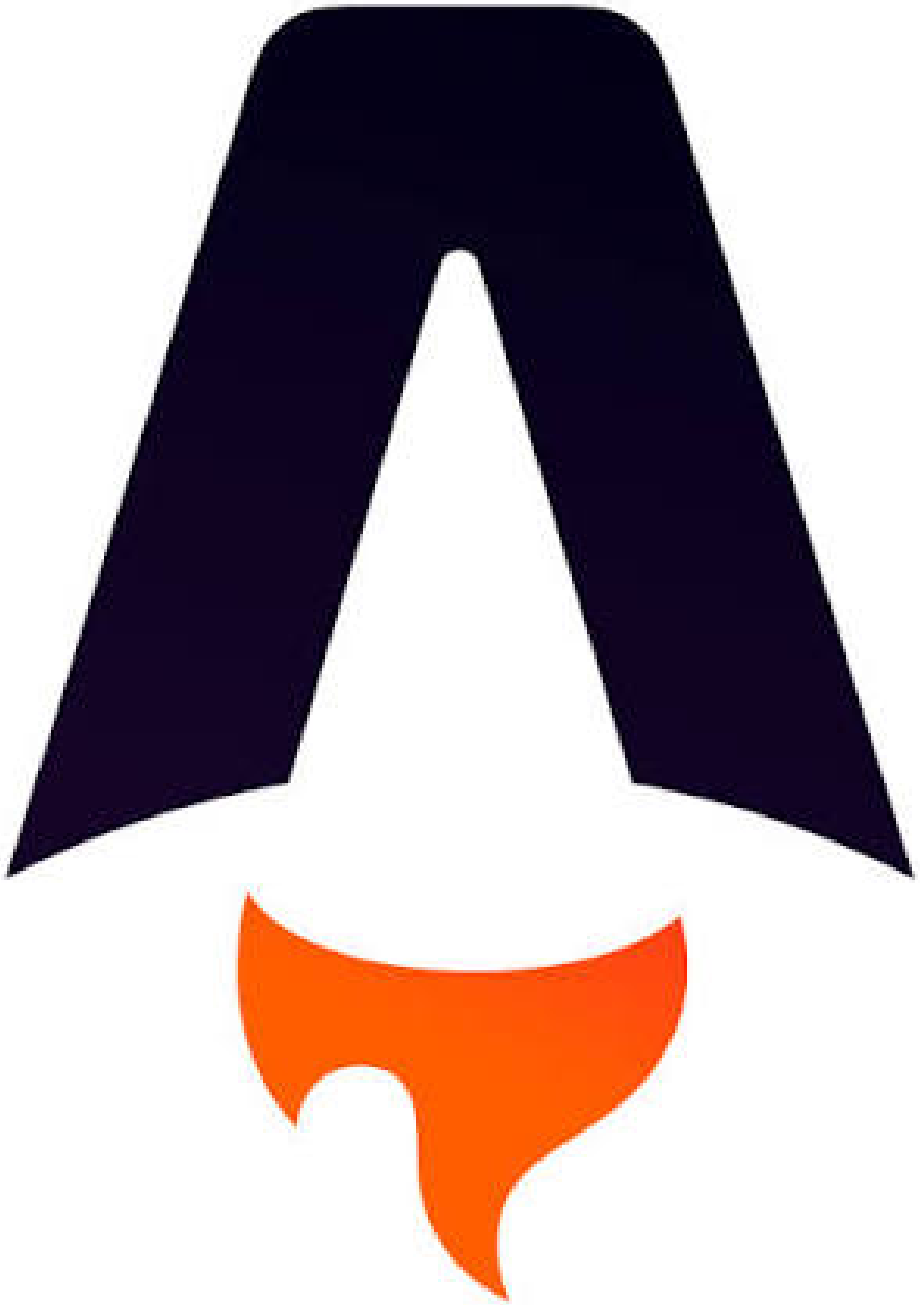
RONNY YARIEL JUÁREZ BORJAS 2DAW

Framework Astro



Contenidos

Contenidos 1
Meta-Framework Astro 2
Características Principales 2
Requisitos e Instalación 3
Uso de Astro 5
Formulario 7
Astro Props 11
Página de Errores 12
Ventajas 13
Recursos 13

Meta-Framework Astro

Astro es un Framework que es utilizado para el desarrollo Web . Se centra en las páginas web de contenido como puede ser el comercio electronico,blogs,marketing entre otros.

Astro sirve para renderizar tu codigo y asi mejorar el rendimiento de tu web , escribes código frontend , este lo procesa todo en el servidor y le manda al usuario HTML y CSS ,reduciendo bastante el código Javascript, esto permite un uso rápido de la web y mejora el SEO de está para aparecer en los primeros resultados de busqueda.

Este Framework fue creado por **Fred K. Schott** junto con un grupo de desarrolladores independientes en 2021. La version más reciente es la **7.3.8**.

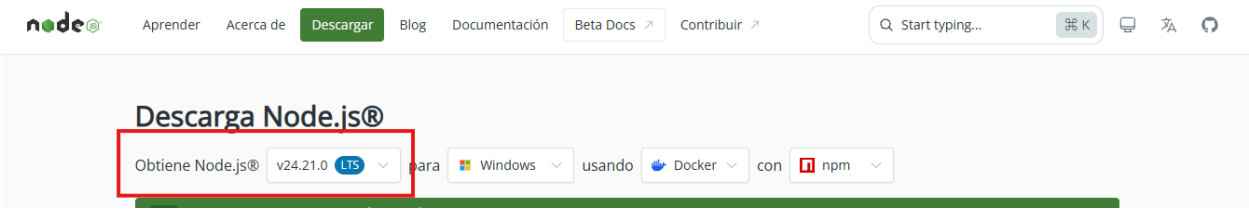


Características Principales

- Astro procesa todos sus componentes en el servidor reduciendo el JavaScript y mandandoles al usuario HTML y CSS.
- Permite añadir un componente dinámico en medio de tu HTML estático como por ejemplo un Bóton.
- Es compatible con una variedad de frameworks populares, React , Svelte , Vue.
- Prioriza el renderizado del lado del servidor sobre el renderizado del lado del cliente en el navegador siempre que sea posible.

Requisitos e Instalación

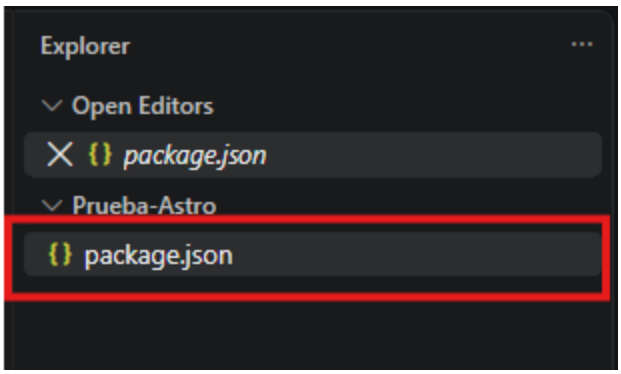
Para poder instalar Astro primero tendremos que instalar **Node.js** para poder utilizar su gestor de paquetes (npm) algo a tener en cuenta es que Astro solo permite las versiones **LTS** de node , las versiones nuevas de Node son muy avanzadas por lo que todavia no cuentan con la compatibilidad con Astro.



Crearemos el archivo packaje.json , este archivo sirve para configurar nuestro proyecto ,identifica el proyecto y gestiona las dependencias del proyecto ,en este caso Astro , ejecutamos elcomando **npm init –yes** en la terminal de VS.code.

```
PS C:\Users\Usuario\Downloads\Prueba-Astro> npm init --yes
Wrote to C:\Users\Usuario\Downloads\Prueba-Astro\package.json:

{
  "name": "prueba-astro",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
```



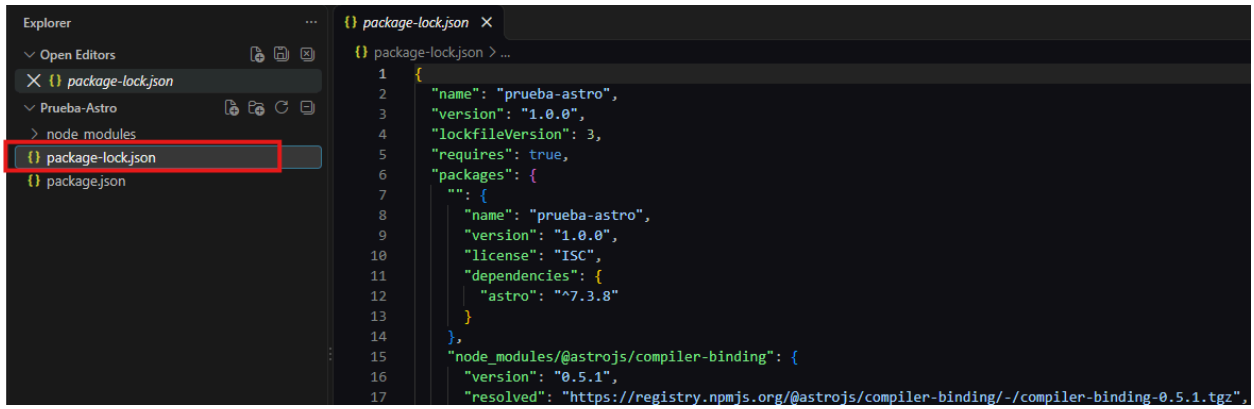
Ahora vamos a installar Astro con el comando **npm install astro ,** esto nos descargara todos los paquetes necesarios y nos generará un archivo json.

```
PS C:\Users\Usuario\Downloads\Prueba-Astro> npm install astro

added 183 packages, and audited 184 packages in 37s

70 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Usuario\Downloads\Prueba-Astro>
```

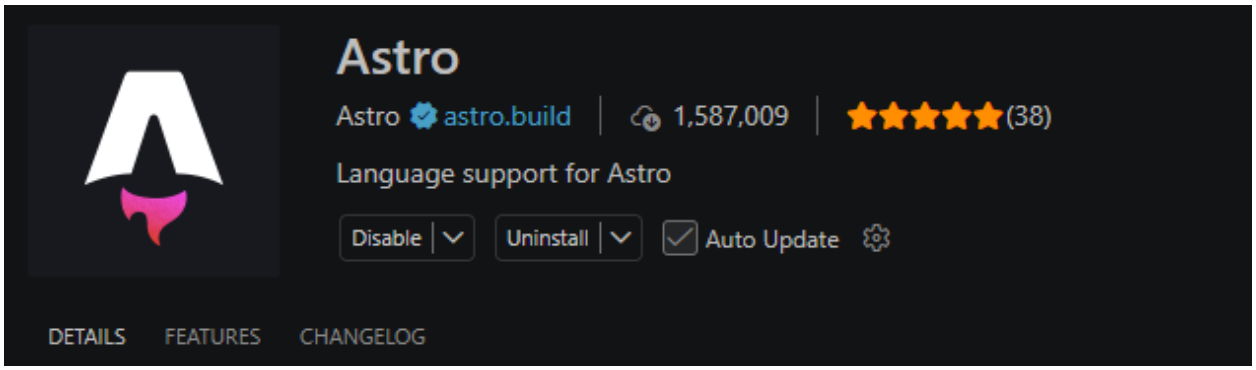


Vamos a indicarle a npm qué comando puede ejecutar en nuestro proyecto de Astro ,esto lo hacemos en package.json en la Parte de scripts.



- dev inicia el servidor de desarrollo para que podamos ver nuestra página mientras programamos.
- build genera la versíon que está en producción para publicarla en un servidor.
- preview nos permite visualizar localmente la producción que hemos generado.

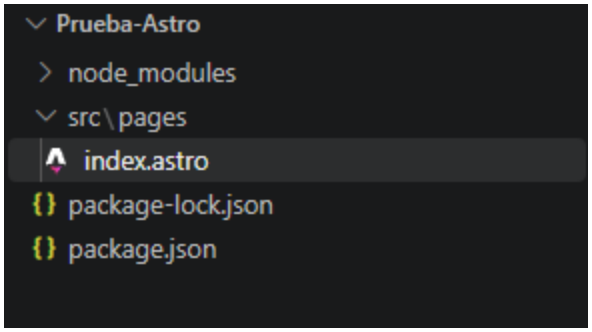
Posteriormente tendremos que instalarlos la extensión de Astro que está disponible para Visual studio Code,



Uso de Astro

Vamos a realizar un ejemplo basico de uso comenzando con un “Hello World” , en este caso tendremos que crear un archivo index.astro en la carpeta de nuestro proyecto .

Primeramente dentro de nuestro proyecto crearemos una carpeta src y dentro de ella otra carpeta llamada pages , donde irá nuestro index.astro, la ruta de los archivo deberia quedar algo asi **C:\Users\Usuario\Downloads\Prueba-Astro\src\pages\index.astro.**



package.json | index.astro | Extension: Astro

src > pages > index.astro > html > head > style > h1

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        h1 {
            color: red;
        }
    </style>
</head>
<body>

    <h1>Hello World</h1>

</body>
```

Ahora vamos a correr nuestro servidor desde la terminal con el comando **npm run dev**

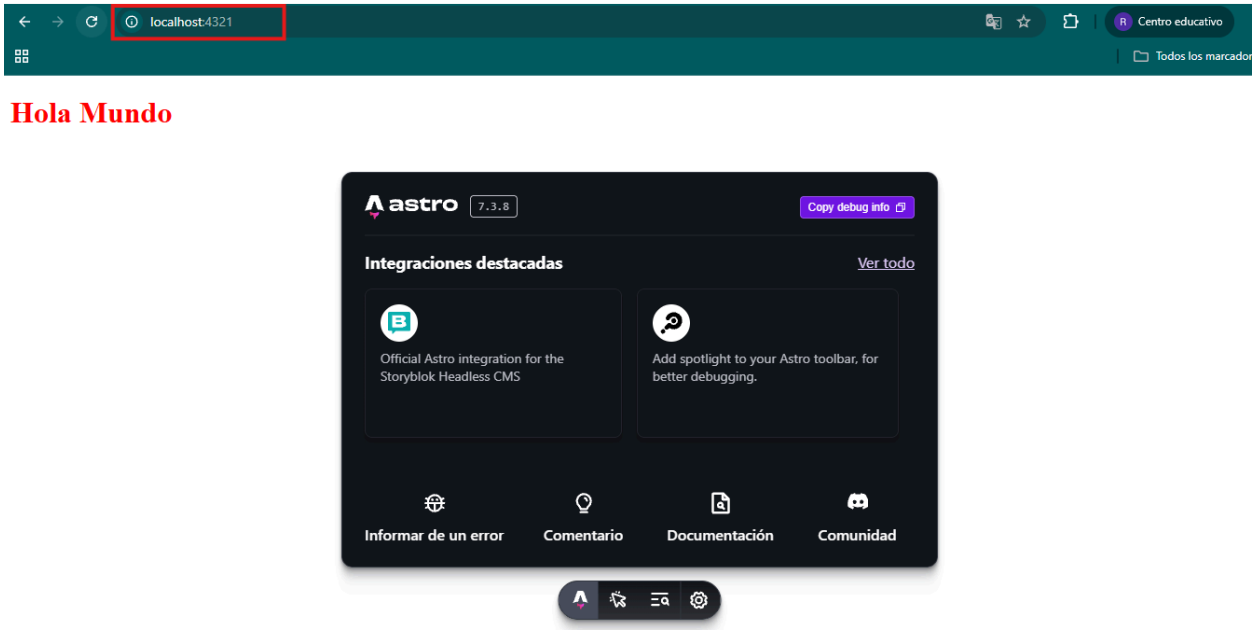
```
PS C:\Users\Usuario\Downloads\Prueba-Astro> npm run dev

> prueba-astro@1.0.0 dev
> astro dev

▶ Astro collects anonymous usage data.
```

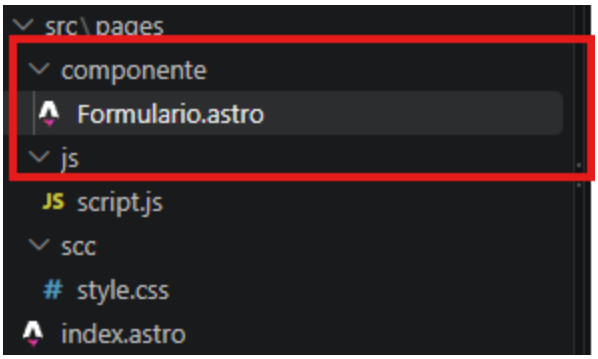
```
15:56:31 [vite] connected.
15:56:31 [types] Generated 2ms
15:56:31 [vite] connected.
astro v7.3.8 ready in 316 ms
Local    http://localhost:4321/
Network  use --host to expose
15:56:31 watching for file changes...
```

Astro nos proporcionará un pequeño menú interactivo en el propio servidor que hemos creado (**Astro Dev Toolbar**) aquí podremos ver la version que tenemos instalada ademas de poder inspeccionar componentes de nuestra web.



Formulario

Vamos a crear un formulario que recoja datos para hacer un ejemplo un poco más completo , en Astro vamos a crear un componente que despues vamos a exportar en nuestro index.astro , crearemos una carpeta componente y dentro de ella estara Formulario.astro (Es importante que empiezan en mayúscula porque sino astro lo tomara como etiqueta html).



```astro
<main id="principal">
    <div class="caja">
    <!-- Formulario -->
    <form>
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre"/>

        <label for="telefono">Teléfono:</label>
        <input type="tel" id="telefono" name="telefono" />

        <label for="direccion">Dirección:</label>
        <input type="text" id="direccion" name="direccion" />

        <label for="email">Correo:</label>
        <input type="email" id="email" name="email"/>

        <div id="boton">

        </div>
    </form>
</div>
```

Una vez que tenemos nuestro componente con los estilos vamos a crear nuestro javaScritp para el funcionamiento.

```javascript
const main = document.getElementById("principal");
const b1 = document.getElementById("boton"); // main del boton

// Creamos boton para generar tarjeta

let boton = document.createElement("button");
boton.textContent = "Enviar";

// agregamos al componente formularios
b1.appendChild(boton);


// creamos la tarjeta de resultados
let tarjeta = document.createElement("div");

```

```
// creamos el evento del botón
boton.onclick = function (){
 // recolectamos los datos
let nombre = document.getElementById("nombre").value;
let telefono = document.getElementById("telefono").value;
let direccion = document.getElementById("direccion").value;
let correo = document.getElementById("email").value;

//agregamos el estilo scc
tarjeta.classList.add("caja");

tarjeta.innerHTML = `Nombre:${nombre}<br>
                    Teléfono:${telefono}<br>
                    Dirección:${direccion}<br>
                    Correo:${correo}<br>`;

    // agregamos al main
    main.appendChild(tarjeta);

}
```

Una vez que ya tenemos terminado nuestro javaScript vamos a agregarlo a nuestro componente formulario.

```
</div>
<script src="C:/Users/Usuario/Downloads/Prueba-Astro/src/pages/js/script.js"></script>
</main>
```

Ahora en nuestro index vamos a importer el componente , es importante que a la hora de importar los pongamos dentro de guiones (- - -) , sino Astro lo interpreta como texto plano o html .Agregamos el nombre del componente y su ruta ,

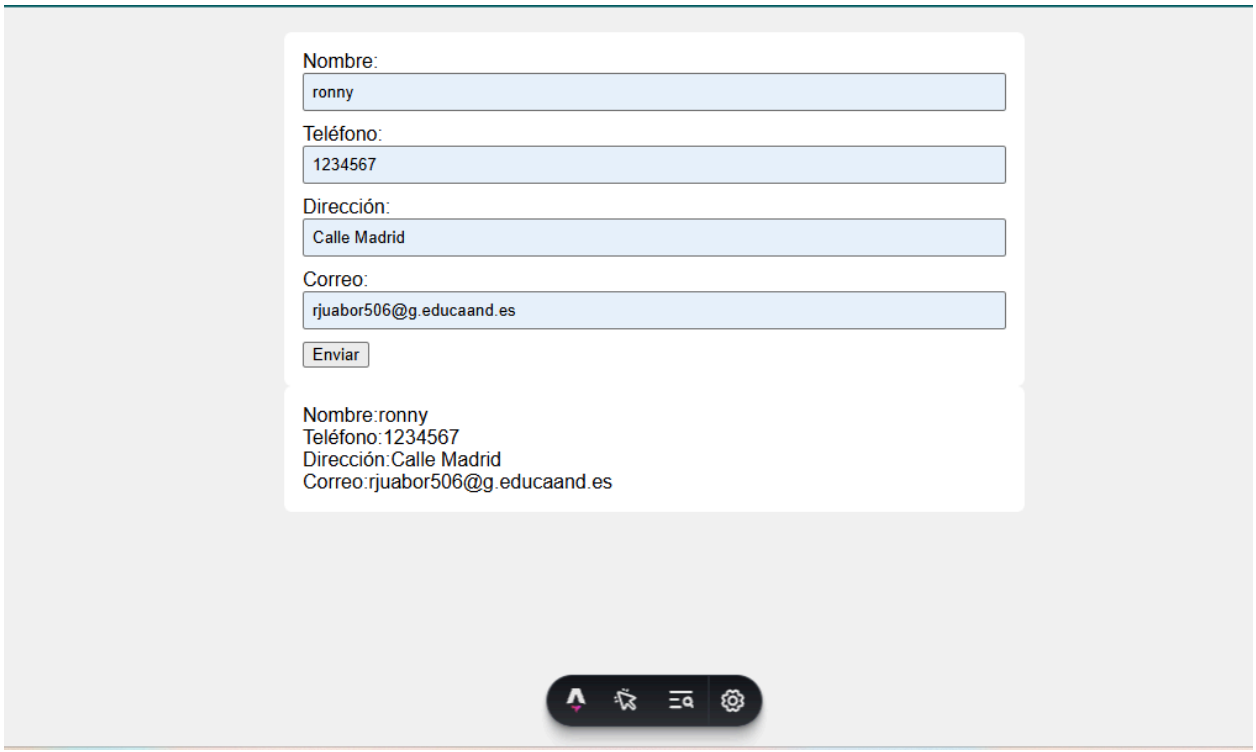
```
1  ---
2  import Formulario from 'C:/Users/Usuario/Downloads/Prueba-Astro/src/pages/componente/formulario.astro';
3
4  import 'C:/Users/Usuario/Downloads/Prueba-Astro/src/pages/scc/style.css';
5  ---
```

Ahora solo tendremos que agregar el componente en el main del html , de esta forma se queda una estructura más limpia a la hora d etrabajar con grandes proyectos.

```
---
import Formulario from 'C:/Users/Usuario/Downloads/Prueba-Astro/src/pages/c

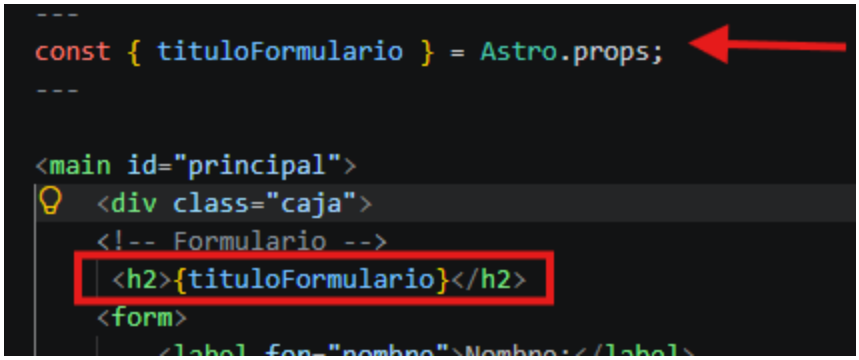
import 'C:/Users/Usuario/Downloads/Prueba-Astro/src/pages/scc/style.css';
---

<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>formulario</title>
</head>
<body>
    <div class="contenedor" >
        <Formulario />
    </div>
</body>
</html>
```



Astro Props

Astro pop nos permitira utilizar reutilizar componentes y que sea dinamicos , por ejemplo el título del formulario , creamos una constante que llama a Astro prosp y luego lo agregamos en el formulario.



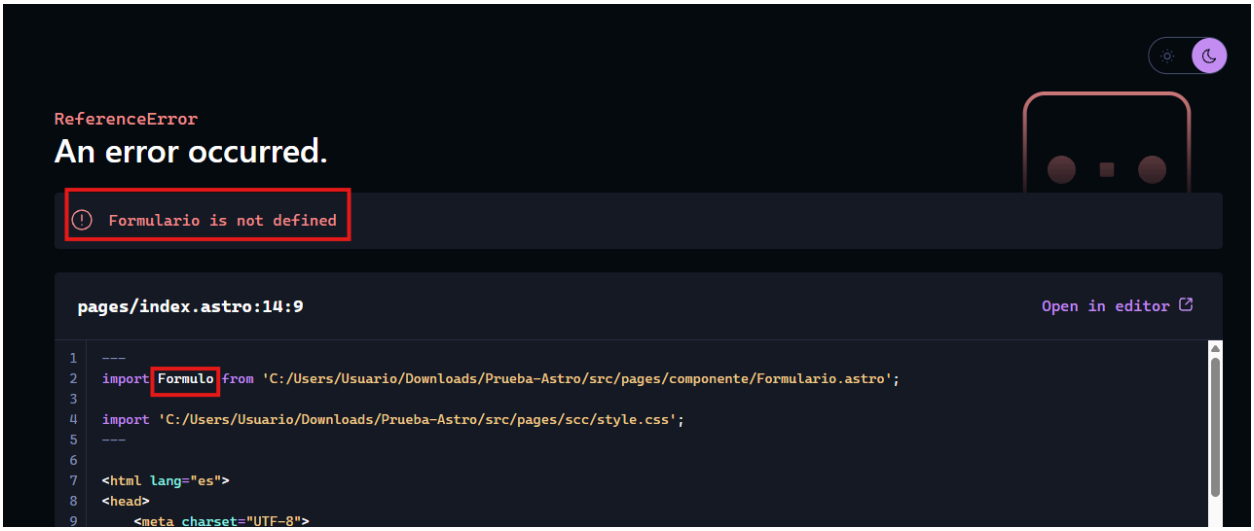
Ahora en el index siempre que llamamos a nuestro componente podremos ponerle un nombre diferente esto facilita el uso de componentes y reduce la repetición de código.

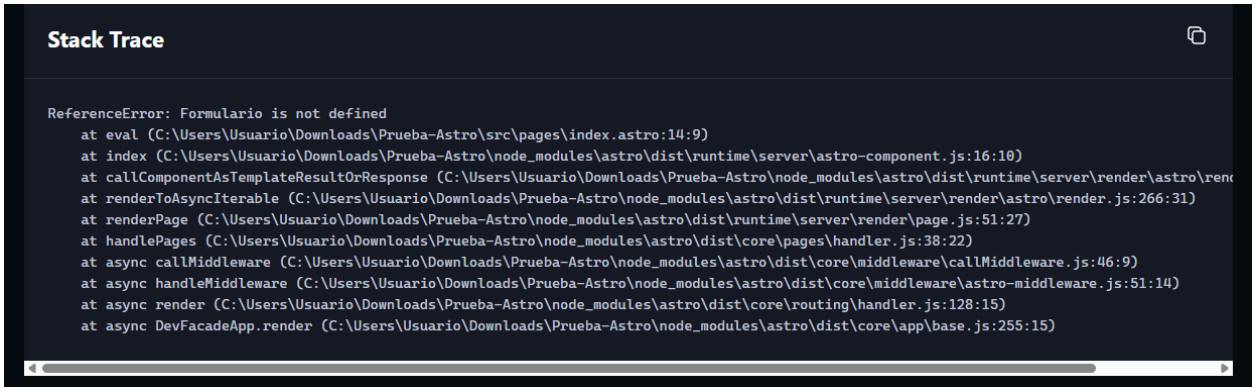




Página de Errores

Astro cuenta con una página de errores en el propio servidor que ellos nos proporcionan , ademas de indicar donde tienes el error, en este caso teniamos un error de nombre de componente.





Ventajas

Personalmente considero que es algo bastante sencillo de aprender , su curva de aprendizaje no es tan elevada , sobretodo por que es un framework que se orienta más en la eficiencia del servidor para que los usuarios gasten menos memoria de sus dispositivos y tengan una mejor experiencia , supongo que esto se mirara más reflejado en proyectos grandes.

Si lo usaría para proyectos donde el usuario no tenga tanta interacción en la web , por ejemplo un blog o promocionar algun producto , para otros desarrollos donde la base de la web es la interacción que tiene en el cliente como puede ser un chat en tiempo real elegiría otras opciones.

En Conclusión, dependiendo en que lo utilicemos Astro puede ser una herramienta bastante buena para utilizar en el desarrollo de nuestra web.

Recursos

- devdocs
- docs.astro
- node
- repositorio